

Reviewer Pack — Minimal Order of Geometric Symmetry and Circuit Monotone Wid...

Entry b43ade6266f2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 06:20:19 UTC

Minimal Order of Geometric Symmetry and Circuit Monotone Width Inequality

Entry ID: b43ade6266f2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 06:20:19 UTC

1. Conjecture

Field A (mathematical branch): Geometric Group Theory **Field B** (complexity object): Boolean Circuit Complexity

Statement:

The minimal order of the symmetry group of a Boolean circuit's gate arrangement, as measured by the largest non-trivial subgroup isomorphic to a geometric group, is monotonically related to the circuit's monotone width.

Rationale (proposer's reasoning):

Geometric group theory provides tools for studying symmetries in discrete structures. By exploring these symmetries, we may uncover new relationships between geometric and computational properties of circuits, potentially leading to novel complexity lower bounds. The minimal order of symmetry groups could offer insights into the intrinsic structure of circuits that are not captured by traditional measures like monotone width.

Taxonomy category: GeometricGroupTheory × BooleanCircuitComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 31604f9fbe22274a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all seeds, the correlation coefficient between the computed minimal order of symmetry group and monotone width exceeds 0.7, with p-value ≤ 0.05 .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "geometric group theory" AND "Boolean circuit complexity" AND symmetry group" - "monotone width" inproceedings arxiv:"math/" - "circuit gate arrangement" AND isomorphic to AND geometric group"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from itertools import combinations

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_circuit(n, m):
        gates = []
        for _ in range(m):
            gate_type = random.choice(['AND', 'OR'])
            inputs = random.sample(range(n), 2)
            gates.append((gate_type, inputs))
        return gates

    def is_subgroup(H, G):
        if not H:
            return True
        if len(H) > len(G):
            return False
        for h in H:
            if h not in G:
                return False
        return all(is_subgroup(generate_circuit(len(h), 1)[0], G) for g in H)

    def find_largest_subgroup(circuit):
        n = len(circuit[0][1])
        elements = list(range(n))
        largest_subgroup = []
        for r in range(1, len(elements)):
            for subset in combinations(elements, r):
                subgroup = generate_circuit(r, 1)[0]
                if is_subgroup(subgroup, circuit):
```

```

        largest_subgroup = subgroup
    return largest_subgroup

def compute_symmetry_group_order(circuit):
    n = len(circuit[0][1])
    elements = list(range(n))
    order = 1
    for r in range(1, len(elements)):
        for subset in combinations(elements, r):
            subgroup = generate_circuit(r, 1)[0]
            if is_subgroup(subgroup, circuit):
                order *= math.factorial(len(subset))
    return order

def compute_monotone_width(circuit):
    n = len(circuit[0][1])
    width = 0
    for i in range(n):
        for j in range(i + 1, n):
            if all(gate[0] == 'AND' and gate[1] == (i, j) for gate in circuit):
                width += 1
    return width

n_values = [5, 10, 15, 20, 30, 40]
instances_tested = 0
total_symmetry_group_order = 0
total_monotone_width = 0
n_max = 0

for n in n_values:
    for _ in range(5):
        circuit = generate_circuit(n, random.randint(2 * n, 3 * n))
        symmetry_group_order = compute_symmetry_group_order(circuit)
        monotone_width = compute_monotone_width(circuit)

        total_symmetry_group_order += symmetry_group_order
        total_monotone_width += monotone_width
        instances_tested += 1
        n_max = max(n_max, n)

mean_symmetry_group_order = total_symmetry_group_order / instances_tested
mean_monotone_width = total_monotone_width / instances_tested

if instances_tested < 30:
    return {
        "metric_name": "symmetry_group_order",
        "metric_value": mean_symmetry_group_order,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "insufficient_instances"
    }

covariance = 0
variance_symmetry_group_order = 0
variance_monotone_width = 0

```

```

for _ in range(5):
    circuit = generate_circuit(n_max, random.randint(2 * n_max, 3 * n_max))
    symmetry_group_order = compute_symmetry_group_order(circuit)
    monotone_width = compute_monotone_width(circuit)

    covariance += (symmetry_group_order - mean_symmetry_group_order) * (monotone_width - mean_monotone_width)
    variance_symmetry_group_order += (symmetry_group_order - mean_symmetry_group_order) ** 2
    variance_monotone_width += (monotone_width - mean_monotone_width) ** 2

covariance /= 5
variance_symmetry_group_order /= 5
variance_monotone_width /= 5

if variance_symmetry_group_order == 0 or variance_monotone_width == 0:
    return {
        "metric_name": "symmetry_group_order",
        "metric_value": mean_symmetry_group_order,
        "instances_tested": instances_tested,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "variance_zero"
    }

correlation_coefficient = covariance / (math.sqrt(variance_symmetry_group_order) * math.sqrt(variance_monotone_width))

return {
    "metric_name": "symmetry_group_order",
    "metric_value": mean_symmetry_group_order,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": correlation_coefficient > 0.7,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    else:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"correlation_coefficient_too_low\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a624539c.py", line 143, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a624539c.py", line 78, in run_trial
    symmetry_group_order = compute_symmetry_group_order(circuit)
                           ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a624539c.py", line 55, in compute_symmetry_group_order
    subgroup = generate_circuit(r, 1)[0]
               ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_a624539c.py", line 24, in generate_circuit
    inputs = random.sample(range(n), 2)
              ~~~~~
```

```
File "/usr/lib/python3.12/random.py", line 430, in sample
    raise ValueError("Sample larger than population or is negative")
```

ValueError: Sample larger than population or is negative

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it was unable to compute the required correlation coefficient and p-value to support or falsify the conjecture. | next: Review the code for potential bugs that could cause a crash during execution and ensure proper error handling. Once fixed, rerun the test with appropriate seeds to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	14901
2	preregistration	ollama_remote	glm4:latest	0	0	11225
3	novelty	ollama_remote	glm4:latest	0	0	8668
4	novelty	ollama_remote	glm4:latest	0	0	9004
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20308
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8464
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	36848

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16820
9	judge	ollama_remote	glm4:latest	0	0	22479

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 148717 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/b43ade6266f2.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/b43ade6266f2.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b43ade6266f2.tar.gz (if generated)